

Subscription & Billing Management Platform

Database Schema Reference — Tables, Use Cases & User Story Coverage

Version 1.0 | Generated March 2026

Overview

This document provides a detailed reference for all 16 database tables in the Subscription & Billing Management Platform. For each table, the document describes its purpose, its column definitions, and — critically — the exact user stories that the table supports. Tables are organised by functional domain to aid navigation.

The platform is designed around five functional domains: Customer & Identity Management, Plan & Subscription Management, Billing Engine, Payments & Collections, and Audit & Compliance (embedded within the identity domain).

Table Summary

Table Name	Domain	Primary Role
CUSTOMER	Customer & Identity	Central customer registry
USER	Customer & Identity	Internal staff with RBAC roles
AUDIT_LOG	Customer & Identity	Immutable action log for compliance
PLAN	Plan & Subscription	Subscription tier definitions
PLAN_FEATURE	Plan & Subscription	Feature entitlements per plan
SUBSCRIPTION	Plan & Subscription	Active customer-plan lifecycle records
SUBSCRIPTION_ADDON	Plan & Subscription	Optional add-on charges per subscription
DISCOUNT	Plan & Subscription	Coupon and discount rule definitions
SUBSCRIPTION_DISCOUNT	Plan & Subscription	Applied discounts per subscription
BILLING_JOB	Billing Engine	Scheduled automated billing run records
INVOICE	Billing Engine	Generated financial billing documents
INVOICE_LINE_ITEM	Billing Engine	Itemised breakdown within an invoice

Table Name	Domain	Primary Role
TAX_RULE	Billing Engine	Region-based tax configuration
PAYMENT	Payments & Collections	Payment transactions against invoices
PAYMENT_METHOD	Payments & Collections	Tokenised customer payment instruments
DUNNING_EVENT	Payments & Collections	Automated retry and reminder events

Customer & Identity Management

CUSTOMER

Central registry for all customers onboarded to the platform.

Column Definitions

Column	Type	Description
id	uuid PK	Unique customer identifier
name	string	Customer or company contact name
email	string	Primary billing email
phone	string	Contact phone number
company	string	Business name
billing_address	string	Address used on invoices
tax_id	string	VAT / GST / tax registration number
status	enum	active suspended churned trial
created_at	timestamp	Account creation time
updated_at	timestamp	Last modification time

User Story Coverage

User Story	How This Table Covers It
As a billing admin, I want to onboard a new customer with their contact and billing details so the system can associate subscriptions and invoices to them.	Stores name, email, phone, billing_address, tax_id, and lifecycle status.
As a finance manager, I want to view a customer's billing profile to resolve invoice disputes quickly.	Provides a single source of truth for all customer identity and contact attributes.
As a compliance officer, I want customer records to carry timestamps so I can audit when accounts were created or modified.	created_at and updated_at support audit traceability.

USER

Internal staff accounts with role-based access control (RBAC). Separate from customers.

Column Definitions

Column	Type	Description
id	uuid PK	Unique staff user identifier
name	string	Staff member full name
email	string	Login email
password_hash	string	Bcrypt / argon2 hashed password
role	enum	admin finance support readonly
is_active	boolean	Whether the account is enabled
last_login	timestamp	Timestamp of most recent login
created_at	timestamp	Account creation time

User Story Coverage

User Story	How This Table Covers It
As a system administrator, I want to assign roles (admin, finance, support) to internal users so each person only accesses what they need.	role enum drives RBAC enforcement across all protected routes.
As a security auditor, I want to track the last login time of each user to detect dormant or compromised accounts.	last_login timestamp is updated on every successful authentication.
As a finance manager, I want to know which staff member generated or edited an invoice for accountability.	USER.id is referenced by AUDIT_LOG.user_id on every sensitive action.

AUDIT_LOG

Immutable log of every sensitive action performed by internal users. Critical for compliance and dispute resolution.

Column Definitions

Column	Type	Description
id	uuid PK	Unique log entry identifier
user_id	uuid FK	References USER who performed the action

Column	Type	Description
entity_type	string	E.g. INVOICE, PAYMENT, SUBSCRIPTION
entity_id	uuid	ID of the affected record
action	enum	CREATE UPDATE DELETE BILLING_RUN
old_values	jsonb	Snapshot of values before change
new_values	jsonb	Snapshot of values after change
ip_address	string	Originating IP address
created_at	timestamp	Time of the action

User Story Coverage

User Story	How This Table Covers It
As a compliance officer, I want every billing run, invoice edit, and payment entry to be logged with actor and timestamp so I can produce a full audit trail.	Records entity_type, entity_id, action, old_values, new_values, and ip_address for each event.
As a system administrator, I want to know the IP address of the user who made a sensitive change to detect unauthorized access.	ip_address is captured on every audit record.
As a finance manager, I want to roll back an incorrect payment entry by reviewing what values were changed.	old_values (jsonb) stores the pre-change state for every audited record.

Plan & Subscription Management

PLAN

Defines available subscription tiers — pricing, billing cadence, and trial configuration.

Column Definitions

Column	Type	Description
id	uuid PK	Unique plan identifier
name	string	Display name (e.g. Pro, Enterprise)
description	string	Marketing description
base_price	decimal	Price per billing cycle
billing_cycle	enum	monthly quarterly annual usage
plan_type	enum	flat per_seat usage_based tiered
trial_days	int	Free trial duration in days
is_active	boolean	Whether plan is open for new subscriptions
created_at	timestamp	Plan creation time

User Story Coverage

User Story	How This Table Covers It
As a product manager, I want to define multiple subscription plans (monthly, annual, usage-based) with distinct prices so customers can self-select the right tier.	billing_cycle enum and base_price cover all billing cadences.
As a billing admin, I want to deactivate a legacy plan without deleting it so existing subscribers are unaffected while no new signups can choose it.	is_active boolean allows soft-deactivation without cascade deletes.
As a sales rep, I want to offer a free trial period on specific plans to reduce signup friction.	trial_days defines the trial window; SUBSCRIPTION.trial_end_date is computed from this.

PLAN_FEATURE

Feature entitlements tied to a plan. Enables feature-gating and transparent plan comparison.

Column Definitions

Column	Type	Description
id	uuid PK	Unique feature row identifier
plan_id	uuid FK	References PLAN
feature_name	string	E.g. api_calls, storage_gb, seats
feature_value	string	Limit or entitlement value
unit	string	Unit of measure (calls, GB, users)

User Story Coverage

User Story	How This Table Covers It
As a product manager, I want each plan to list specific feature limits (e.g. API calls: 10,000/month) so the billing system can enforce entitlements.	feature_name, feature_value, and unit define each limit per plan.
As a customer success manager, I want to display a feature comparison table on the pricing page using live plan data.	PLAN_FEATURE rows can be queried per plan to build comparison matrices.

SUBSCRIPTION

Lifecycle record for each customer's active plan relationship. The central hub of the billing engine.

Column Definitions

Column	Type	Description
id	uuid PK	Unique subscription identifier
customer_id	uuid FK	References CUSTOMER
plan_id	uuid FK	References PLAN
status	enum	active trial paused cancelled past_due
start_date	date	Subscription activation date
end_date	date	Cancellation or expiry date
trial_end_date	date	End of trial window
next_billing_date	date	Date the next invoice is due

Column	Type	Description
current_mrr	decimal	Monthly recurring revenue from this subscription
quantity	int	Number of seats or units
created_at	timestamp	Record creation time
updated_at	timestamp	Last status change time

User Story Coverage

User Story	How This Table Covers It
As a billing engine, I want to know which subscriptions are due for renewal today so I can trigger invoicing automatically.	next_billing_date is the primary field queried by the daily BILLING_JOB.
As a customer success manager, I want to pause or cancel a subscription without losing historical data.	status enum (active paused cancelled trial past_due) manages the lifecycle.
As a finance analyst, I want to calculate MRR by summing current_mrr across all active subscriptions.	current_mrr is maintained per subscription and updated on plan changes or proration.
As a billing admin, I want to track trial periods separately so I can trigger conversion reminders before trial_end_date.	trial_end_date is set from PLAN.trial_days on subscription creation.

SUBSCRIPTION_ADDON

Optional add-ons purchased on top of a base plan (e.g. extra storage, priority support).

Column Definitions

Column	Type	Description
id	uuid PK	Unique add-on record identifier
subscription_id	uuid FK	References SUBSCRIPTION
addon_name	string	Name of the add-on service
price	decimal	Price per billing cycle
quantity	int	Number of add-on units

User Story Coverage

User Story	How This Table Covers It
As a customer, I want to add extra storage to my plan without upgrading my entire tier.	Each add-on has its own addon_name, price, and quantity tied to the subscription.
As a billing engine, I want to include all active add-ons in the invoice calculation for a subscription.	INVOICE_LINE_ITEM is generated for each SUBSCRIPTION_ADDON on billing run.

DISCOUNT

Master coupon/discount registry. Defines discount rules that can be applied to subscriptions.

Column Definitions

Column	Type	Description
id	uuid PK	Unique discount identifier
code	string	Human-readable coupon code
discount_type	enum	flat percentage
value	decimal	Discount amount or percentage
valid_from	date	Discount activation date
valid_until	date	Discount expiry date
max_uses	int	Maximum allowed redemptions
used_count	int	Current redemption count
is_active	boolean	Whether discount is currently active

User Story Coverage

User Story	How This Table Covers It
As a sales rep, I want to create time-limited discount codes for promotional campaigns so customers can redeem reduced pricing.	valid_from, valid_until, and code fields manage coupon lifecycle.
As a billing admin, I want to cap how many times a discount code can be used to prevent revenue leakage.	max_uses and used_count enforce redemption limits.
As a finance manager, I want discounts to be flat-amount or percentage-based to accommodate different promotional structures.	discount_type enum (flat percentage) and value field handle both cases.

SUBSCRIPTION_DISCOUNT

Junction table linking a discount to a subscription. Tracks when and how long a discount applies.

Column Definitions

Column	Type	Description
id	uuid PK	Unique application record
subscription_id	uuid FK	References SUBSCRIPTION
discount_id	uuid FK	References DISCOUNT
applied_at	date	When the discount was first applied
expires_at	date	When the discount expires for this subscription

User Story Coverage

User Story	How This Table Covers It
As a billing engine, I want to know if a subscription has an active discount at billing time so I can apply it to the invoice automatically.	applied_at and expires_at determine if the discount is valid at invoice generation time.
As a finance analyst, I want to see which subscriptions received discounts so I can calculate effective revenue versus list price.	Joining SUBSCRIPTION_DISCOUNT with DISCOUNT reveals discount history per subscription.

Billing Engine

BILLING_JOB

Represents a scheduled automated billing run. Tracks execution status, volume, and errors.

Column Definitions

Column	Type	Description
id	uuid PK	Unique billing job identifier
scheduled_date	date	Date the job was scheduled to run
status	enum	pending running completed failed
subscriptions_processed	int	Count of subscriptions evaluated
invoices_generated	int	Count of invoices created
total_amount	decimal	Sum of all invoices in this run
started_at	timestamp	Actual execution start time
completed_at	timestamp	Execution completion time
error_log	string	Error details if job failed

User Story Coverage

User Story	How This Table Covers It
As a billing engineer, I want a daily job to automatically identify subscriptions due for renewal and generate invoices without manual intervention.	scheduled_date triggers the run; status (pending running completed failed) tracks progress.
As a finance manager, I want to see a summary of each billing run — how many subscriptions were processed and total amount billed.	subscriptions_processed, invoices_generated, and total_amount are updated on completion.
As a support engineer, I want to inspect error logs when a billing run fails so I can fix and re-run specific subscriptions.	error_log (text) captures failure reasons per run.

INVOICE

Financial document generated per billing cycle per subscription. The authoritative billing record.

Column Definitions

Column	Type	Description
id	uuid PK	Unique invoice identifier
subscription_id	uuid FK	References SUBSCRIPTION
billing_job_id	uuid FK	References BILLING_JOB that created this invoice
invoice_number	string	Human-readable invoice reference (e.g. INV-2024-00123)
issue_date	date	Date invoice was generated
due_date	date	Payment deadline
subtotal	decimal	Sum before discounts and taxes
discount_amount	decimal	Total discount applied
tax_amount	decimal	Total tax applied
total_amount	decimal	Final amount due
status	enum	draft sent paid overdue void
notes	string	Additional billing notes
created_at	timestamp	Invoice creation time

User Story Coverage

User Story	How This Table Covers It
As a customer, I want to receive an invoice with a unique number, issue date, due date, and itemised breakdown after each billing cycle.	invoice_number, issue_date, due_date, and line items (via INVOICE_LINE_ITEM) compose the invoice.
As a finance manager, I want invoices to reflect applied discounts and taxes as separate line amounts for transparency.	discount_amount, tax_amount, and subtotal are stored independently to support line-level reconciliation.
As a billing admin, I want to track invoice status (draft, sent, paid, overdue, void) to manage collections efficiently.	status enum drives the dunning workflow and payment matching.
As a finance analyst, I want to identify invoices associated with a specific billing run for run-level reconciliation.	billing_job_id links every invoice back to the job that created it.

INVOICE_LINE_ITEM

Itemised breakdown within an invoice. Each plan charge, add-on, or adjustment is a separate line.

Column Definitions

Column	Type	Description
id	uuid PK	Unique line item identifier
invoice_id	uuid FK	References INVOICE
description	string	Charge description (e.g. Pro Plan — October 2024)
quantity	int	Units billed
unit_price	decimal	Price per unit
discount_amount	decimal	Discount applied to this line
line_total	decimal	Net amount for this line (qty * price - discount)

User Story Coverage

User Story	How This Table Covers It
As a customer, I want my invoice to show a line-by-line breakdown of charges (base plan, add-ons, proration) so I can verify what I am billed for.	Each INVOICE_LINE_ITEM has description, quantity, unit_price, discount_amount, and line_total.
As a finance analyst, I want to aggregate line item data across invoices to understand revenue by product type.	description and unit_price enable categorisation and revenue attribution at item level.

TAX_RULE

Region-based tax configuration used to calculate tax on invoices consistently.

Column Definitions

Column	Type	Description
id	uuid PK	Unique tax rule identifier
name	string	Rule name (e.g. Karnataka GST 18%)
region	string	Country or state code (e.g. IN-KA, US-CA)
rate	decimal	Tax rate as a decimal (e.g. 0.18 for 18%)
tax_type	enum	GST VAT sales_tax service_tax
is_active	boolean	Whether the rule is currently in use

User Story Coverage

User Story	How This Table Covers It
As a finance manager, I want the billing engine to automatically apply the correct tax rate based on the customer's region so invoices are always tax-compliant.	region and rate are matched against the customer's billing_address at invoice generation.
As a compliance officer, I want tax rules to be versioned (via is_active) so I can update rates without losing historical calculation context.	is_active allows new rules to be added while preserving old ones for retrospective audits.

Payments & Collections

PAYMENT

Records every payment transaction against an invoice, including gateway response for reconciliation.

Column Definitions

Column	Type	Description
id	uuid PK	Unique payment record identifier
invoice_id	uuid FK	References INVOICE being settled
payment_method_id	uuid FK	References PAYMENT_METHOD used
amount	decimal	Amount paid
status	enum	pending success failed refunded
transaction_id	string	Gateway-issued transaction reference
gateway_response	string	Raw gateway response payload
paid_at	timestamp	Time payment was confirmed
created_at	timestamp	Record creation time

User Story Coverage

User Story	How This Table Covers It
As a billing admin, I want each payment to be linked to the invoice it settles so I can track outstanding receivables accurately.	invoice_id links the payment to its source invoice; status updates the invoice's collection state.
As a finance manager, I want to store the payment gateway's transaction ID so I can reconcile payments with bank statements.	transaction_id and gateway_response store the external reference for reconciliation.
As a support agent, I want to see the exact timestamp a payment was received to respond to customer queries about payment confirmation.	paid_at captures the exact moment of successful payment confirmation.

PAYMENT_METHOD

Stores tokenised payment methods (cards, bank accounts) per customer for automated charging.

Column Definitions

Column	Type	Description
id	uuid PK	Unique payment method identifier
customer_id	uuid FK	References CUSTOMER who owns this method
method_type	enum	card bank_transfer upi wallet
provider_token	string	Tokenised reference from payment gateway
last_four	string	Last 4 digits of card for display
expiry_date	date	Card expiry date
is_default	boolean	Whether this is the primary charge method
is_active	boolean	Whether the method is valid for use
created_at	timestamp	Record creation time

User Story Coverage

User Story	How This Table Covers It
As a customer, I want to save multiple payment methods and designate one as default so recurring billing charges the right card automatically.	is_default and is_active flags manage default selection and card lifecycle.
As a billing engine, I want to charge the customer's default payment method during automated billing runs without requiring manual intervention.	is_default = true is queried during BILLING_JOB to select the charge target.
As a security officer, I want payment details to be stored as provider tokens (not raw card numbers) to maintain PCI-DSS compliance.	provider_token stores the vault reference; last_four is the only PAN fragment retained.

DUNNING_EVENT

Tracks automated payment retry and reminder events for overdue invoices. Powers the collections workflow.

Column Definitions

Column	Type	Description
id	uuid PK	Unique dunning event identifier

Column	Type	Description
invoice_id	uuid FK	References overdue INVOICE
event_type	enum	email_reminder retry_charge suspension_warning final_notice
attempt_number	int	Retry sequence number (1, 2, 3...)
scheduled_at	timestamp	When the event is scheduled to execute
sent_at	timestamp	When the event was actually dispatched
status	enum	pending sent failed skipped

User Story Coverage

User Story	How This Table Covers It
As a billing system, I want to automatically schedule reminder emails and retry attempts when an invoice is overdue to reduce manual collections effort.	event_type (email_reminder retry_charge suspension_warning) and scheduled_at automate the dunning sequence.
As a finance manager, I want to know how many retry attempts have been made for an overdue invoice so I can decide when to escalate to manual collections.	attempt_number increments with each retry event, capped by configured dunning policy.
As a support agent, I want to see when the last dunning communication was sent to a customer so I don't send duplicate messages.	sent_at records actual dispatch time; status (sent failed skipped) shows delivery outcome.

Cross-Table Relationship Summary

The diagram below summarises how the five functional domains interact through foreign key relationships.

Relationship	From Table	To Table	Cardinality	Business Meaning
Customer owns subscriptions	CUSTOMER	SUBSCRIPTION	1 to many	A customer can hold multiple active or historical subscriptions.
Customer owns payment methods	CUSTOMER	PAYMENT_METHOD	1 to many	A customer can save multiple cards or bank accounts.
Plan applied to subscriptions	PLAN	SUBSCRIPTION	1 to many	Many customers can subscribe to the same plan.
Plan has features	PLAN	PLAN_FEATURE	1 to many	Feature entitlements are defined per plan.
Subscription has add-ons	SUBSCRIPTION	SUBSCRIPTION_ADDON	1 to many	Extra charges on top of the base plan.
Subscription uses discounts	SUBSCRIPTION	SUBSCRIPTION_DISCOUNT	many to many (via junction)	Multiple discounts may apply to one subscription over time.
Billing job produces invoices	BILLING_JOB	INVOICE	1 to many	A single billing run generates invoices for all due subscriptions.
Subscription generates invoices	SUBSCRIPTION	INVOICE	1 to many	Each billing cycle produces one

Relationship	From Table	To Table	Cardinality	Business Meaning
				invoice per subscription.
Invoice contains line items	INVOICE	INVOICE_LINE_ITEM	1 to many	Each charge component is a separate line item.
Invoice settled by payments	INVOICE	PAYMENT	1 to many	Partial or full payments can settle an invoice.
Invoice triggers dunning	INVOICE	DUNNING_EVENT	1 to many	Overdue invoices trigger a sequence of retry and reminder events.
Payment uses payment method	PAYMENT_METHOD	PAYMENT	1 to many	The same saved card can be charged across multiple invoices.
Tax rule applied to invoice	TAX_RULE	INVOICE	many to many	One or more tax rules may apply depending on the customer region.
User generates audit log	USER	AUDIT_LOG	1 to many	Every internal user action is recorded in the audit trail.